

ООП (отношения между классами)

Жизнь полна повторяющихся закономерностей, взаимосвязей и иерархий между объектами. Изучая и понимая их, мы можем получить представление о том, как ведут себя объекты в реальной жизни, и лучше понять их.

Например, представьте, что однажды вы идёте по улице и видите ярко-жёлтый предмет, прикреплённый к зелёному кустарнику. Вы, вероятно, догадаетесь, что ярко-жёлтый предмет — это цветок, а зелёный кустарник — это растение. Даже если вы никогда раньше не видели этот вид растений, вы будете знать, что зелёные части — это листья, которые собирают солнечный свет. Вы будете знать, что цветок помогает растению размножаться. Вы бы также знали, что если бы вы убили растение, цветок тоже бы погиб.

Но как вы можете знать всё это, если никогда раньше не сталкивались с растениями такого типа? Вы знаете это, потому что понимаете абстрактное представление о растениях и осознаёте, что это растение является воплощением этой абстракции. Вы знаете, что большинство растений состоят (частично) из листьев, а у некоторых есть цветы. Вы знаете, что листья взаимодействуют с солнечным светом (даже если вы не знаете, как именно), а существование цветка зависит от растения. Поскольку вы знаете всё это о растениях в целом, вы можете многое сказать об этом растении.

Точно так же в программировании есть повторяющиеся шаблоны, взаимосвязи и иерархии. В частности, когда речь идёт о программируемых объектах, те же шаблоны, которые управляют объектами в реальной жизни, применимы и к программируемым объектам, которые мы создаём сами. Изучив их более подробно, мы сможем лучше понять, как повысить удобство повторного использования кода и писать более расширяемые классы.

В предыдущих главах мы уже рассмотрели некоторые идеи, связанные с повторяющимися шаблонами: мы создали циклы и функции, которые позволяют выполнять определённую задачу много раз. Кроме того, мы создали собственные перечисления, структуры и классы, которые позволяют создавать объекты заданного типа.

Мы также рассмотрели некоторые примитивные формы иерархии, такие как массивы (которые позволяют группировать элементы в более крупную структуру) и рекурсию, когда функция вызывает производную версию самой себя.

Однако мы ещё не уделяли особого внимания взаимосвязи между объектами, особенно в контексте программирования.

Отношения между объектами

В реальной жизни между двумя объектами может быть множество различных отношений, и для описания этих отношений мы используем специальные слова «типа отношения». Например: квадрат — это фигура. У автомобиля есть руль. Программист использует клавиатуру. Цветок зависит от опыления пчёлами. Ученик является элементом класса. И ваш мозг существует как «часть» вас (по крайней мере, мы можем обоснованно предположить это, если вы зашли так далеко).

Все эти типы отношений имеют полезные аналогии в C++.

В этой главе мы рассмотрим нюансы типов отношений «является частью», «имеет», «использует», «зависит от» и «является элементом» и покажем, как они могут быть полезны в контексте классов C++. Мы также рассмотрим несколько смежных тем, которые не вписываются в другие главы.

Затем мы посвятим следующие две главы изучению отношений «является» с помощью модели наследования C++ и виртуальных функций. Да, это важно.

Композиция

В реальной жизни сложные объекты часто состоят из более мелких и простых объектов. Например, автомобиль состоит из металлического каркаса, двигателя, нескольких шин, коробки передач, рулевого колеса и множества других деталей. Персональный компьютер состоит из процессора, материнской платы, нескольких модулей памяти и т. д. Даже вы состоите из более мелких частей: у вас есть голова, тело, несколько ног, рук и так далее. Этот процесс создания сложных объектов из более простых называется композицией объектов.

В широком смысле композиция объектов моделирует отношение «иметь» между двумя объектами. У автомобиля есть трансмиссия. У вашего компьютера есть центральный процессор. У вас есть сердце. Сложный объект иногда называют целым или родительским. Более простой объект часто называют частью, дочерним или компонентом.

В C++ вы уже видели, что структуры и классы могут содержать элементы данных различных типов (например, базовые типы или другие классы). Когда мы создаём классы с элементами данных, мы, по сути, конструируем сложный объект из более простых частей, что называется объектной композицией. По этой причине структуры и классы иногда называют составными типами.

Композиция объектов полезна в контексте C++, поскольку позволяет создавать сложные классы, объединяя более простые и удобные в управлении части. Это снижает сложность и позволяет писать код быстрее и с меньшим количеством ошибок, поскольку мы можем повторно использовать код, который уже был написан, протестирован и проверен на работоспособность.

Типы объектной композиции

Существует два основных типа композиции объектов: композиция и агрегация. В этом уроке мы рассмотрим композицию, а в следующем — агрегацию.

Примечание по терминологии: термин «композиция» часто используется для обозначения как композиции, так и агрегации, а не только подтипа композиции. В этом руководстве мы будем использовать термин «композиция объектов», когда будем говорить о том и другом, и «композиция», когда будем говорить конкретно о подтипе композиции.

Состав

Чтобы считаться составом, объект и его часть должны иметь следующие отношения:

- Часть (элемент) является частью объекта (класса)
- Часть (элемент) может принадлежать только одному объекту (классу) одновременно
- Часть (элемент) существует под управлением объекта (класса).
- Часть (элемент) не знает о существовании объекта (класса)

Хорошим примером композиции в реальной жизни являются отношения между телом человека и его сердцем. Давайте рассмотрим их подробнее.

Отношения «часть-целое» — это отношения, при которых часть должна составлять часть целого объекта. Например, сердце является частью тела человека. Часть в составе композиции может быть частью только одного объекта одновременно. Сердце, являющееся частью тела одного человека, не может одновременно быть частью тела другого человека.

В отношениях композиции объект отвечает за существование частей. Чаще всего это означает, что часть создаётся при создании объекта и уничтожается при уничтожении объекта. Но в более широком смысле это означает, что объект управляет временем жизни части таким образом, что пользователю объекта не нужно вмешиваться. Например, при создании тела создаётся и сердце. При уничтожении тела человека уничтожается и его сердце. Из-за этого композицию иногда называют «отношениями смерти».

И, наконец, часть не знает о существовании целого. Ваше сердце работает, не подозревая, что является частью более крупной структуры. Мы называем это однонаправленной связью, потому что тело знает о сердце, но не наоборот.

Обратите внимание, что состав ничего не говорит о переносимости частей. Сердце можно пересадить из одного тела в другое. Однако даже после пересадки оно по-прежнему соответствует требованиям для состава (сердце теперь принадлежит реципиенту и может быть частью только объекта реципиента, если его снова не пересадят).

Наш вездесущий класс фракций - отличный пример композиции:

```
class Fraction
{
private:
    int m_numerator;
    int m_denominator;

public:
    Fraction(int numerator=0, int denominator=1)
        : m_numerator{ numerator }, m_denominator{ denominator }
    {
    }
};
```

У этого класса есть два элемента данных: числитель и знаменатель. Числитель и знаменатель являются частью дроби (содержатся в ней). Они не могут принадлежать более чем одной дроби одновременно. Числитель и знаменатель не знают, что они являются частью дроби, они просто содержат целые числа. При создании экземпляра дроби создаются числитель и знаменатель. При уничтожении экземпляра дроби уничтожаются числитель и знаменатель.

В то время как композиция объектов моделирует отношения типа «имеет-часть» (у тела есть сердце, у дроби есть знаменатель), мы можем быть более точными и сказать, что композиция моделирует отношения «часть-целое» (сердце является частью тела, числитель является частью дроби). Композиция часто используется для моделирования физических отношений, когда один объект физически находится внутри другого.

Части объекта могут быть единичными или множественными. Например, сердце — это единичная часть тела, но тело состоит из 10 пальцев (которые можно смоделировать как массив).

Реализующие композиции

Композиции — один из самых простых для реализации типов отношений в C++. Обычно они создаются как структуры или классы с обычными элементами данных. Поскольку эти элементы данных существуют непосредственно как часть структуры/класса, их время жизни привязано к времени жизни самого экземпляра класса.

Композиции, которым необходимо динамическое выделение или освобождение памяти, могут быть реализованы с помощью указателей на элементы данных. В этом случае класс композиции должен самостоятельно (а не пользователь класса) отвечать за управление памятью.

В целом, если вы можете создать класс с помощью композиции, вы должны создать класс с помощью композиции. Классы, созданные с помощью композиции, просты, гибки и надёжны (поскольку они хорошо очищают себя).

Еще примеры

Во многих играх и симуляторах есть существа или объекты, которые перемещаются по игровому полю, карте или экрану. У всех этих существ/объектов есть одна общая черта: у них есть местоположение. В этом примере мы создадим класс существа, который использует класс точек для хранения местоположения существа.

Для начала давайте создадим класс точек. Наше существо будет жить в двухмерном мире, поэтому у нашего класса точек будет два измерения: X и Y. Мы будем считать, что мир состоит из отдельных квадратов, поэтому эти измерения всегда будут целыми числами.

```
#ifndef POINT2D_H
#define POINT2D_H

#include <iostream>

class Point2D
{
private:
    int m_x;
    int m_y;

public:
    // A default constructor
    Point2D()
        : m_x{ 0 }, m_y{ 0 }
    {
    }

    // A specific constructor
    Point2D(int x, int y)
        : m_x{ x }, m_y{ y }
    {
    }

    // An overloaded output operator
    friend std::ostream& operator<<(std::ostream& out, const Point2D& point)
    {
        out << '(' << point.m_x << ", " << point.m_y << ')';
        return out;
    }

    // Access functions
    void setPoint(int x, int y)
    {
        m_x = x;
        m_y = y;
    }
};

#endif
```

Обратите внимание, что поскольку мы реализовали все наши функции в заголовочном файле (чтобы сократить объём примера), файл Point2D.cpp отсутствует.

Этот класс Point2D состоит из двух частей: значения x и y являются частью Point2D, и срок их службы привязан к сроку службы конкретного экземпляра Point2D.

Теперь давайте создадим наше Существо. У нашего Существа будет несколько свойств: имя, которое будет строкой, и местоположение, которое будет нашим классом Point2D.

```
#ifndef CREATURE_H
#define CREATURE_H

#include <iostream>
#include <string>
#include <string_view>
#include "Point2D.h"
```

```

class Creature
{
private:
    std::string m_name;
    Point2D m_location;

public:
    Creature(std::string_view name, const Point2D& location)
        : m_name{ name }, m_location{ location }
    {
    }

    friend std::ostream& operator<<(std::ostream& out, const Creature& creature)
    {
        out << creature.m_name << " is at " << creature.m_location;
        return out;
    }

    void moveTo(int x, int y)
    {
        m_location.setPoint(x, y);
    }
};
#endif

```

Это Существо также состоит из частей. У названия и местоположения Существа есть один родитель, и продолжительность его жизни связана с продолжительностью жизни Существа, частью которого оно является.

Основной файл программы:

```

#include <string>
#include <iostream>
#include "Creature.h"
#include "Point2D.h"

int main()
{
    std::cout << "Enter a name for your creature: ";
    std::string name;
    std::cin >> name;
    Creature creature{ name, { 4, 7 } };

    while (true)
    {
        // print the creature's name and location
        std::cout << creature << '\n';

        std::cout << "Enter new X location for creature (-1 to quit): ";
        int x{ 0 };
        std::cin >> x;
        if (x == -1)
            break;

        std::cout << "Enter new Y location for creature (-1 to quit): ";
        int y{ 0 };
        std::cin >> y;
        if (y == -1)
            break;

        creature.moveTo(x, y);
    }
}

```

```
    return 0;
}
```

Вот расшифровка этого запускаемого кода:

```
Enter a name for your creature:Dog

Dog is at (4, 7)
Enter new X location for creature (-1 to quit):5

Enter new Y location for creature (-1 to quit):8

Dog is at (5, 8)
Enter new X location for creature (-1 to quit):-1

Process finished with exit code 0
```

Варианты по теме композиции

Хотя большинство композиций напрямую создают свои части при создании композиции и напрямую уничтожают свои части при уничтожении композиции, есть некоторые варианты композиций, которые немного отклоняются от этих правил.

Например:

- Композиция может отложить создание некоторых частей до тех пор, пока они не понадобятся. Например, класс строк может не создавать динамический массив символов до тех пор, пока пользователь не присвоит строке какие-либо данные.
- Композиция может предпочесть использовать часть, предоставленную ей в качестве входных данных, а не создавать её самостоятельно.
- Композиция может делегировать уничтожение своих частей какому-либо другому объекту (например, процедуре сборки мусора).

Ключевой момент здесь в том, что композиция должна управлять своими частями без необходимости для пользователя композиции управлять чем-либо.

Состав и участники класса

Один из вопросов, который часто задают начинающие программисты, когда речь заходит о композиции объектов: «Когда следует использовать элемент класса вместо прямой реализации функции?». Например, вместо того, чтобы использовать класс Point2D для реализации местоположения существа, мы могли бы просто добавить 2 целых числа в класс Creature и написать код в классе Creature для обработки позиционирования. Однако создание собственного класса Point2D (и элемента класса Creature) имеет ряд преимуществ:

1. Каждый отдельный класс может быть относительно простым и понятным, ориентированным на выполнение одной задачи. Это упрощает написание и понимание таких классов, поскольку они более целенаправленны. Например, Point2D заботится только о том, что связано с точками, что упрощает его.
2. Каждый класс может быть автономным, что делает их пригодными для повторного использования. Например, мы могли бы повторно использовать наш класс Point2D в совершенно другом приложении. Или, если бы нашему существу понадобилась ещё одна точка (например, пункт назначения, к которому оно пытается добраться), мы могли бы просто добавить ещё одну переменную-элемент Point2D.
3. Внешний класс может поручить элементам класса большую часть тяжёлой работы, а сам сосредоточиться на координации потока данных между элементами. Это помогает снизить общую сложность внешнего класса, поскольку он может делегировать задачи своим элементам, которые уже знают, как выполнять эти задачи. Например, когда мы перемещаем наше существо, оно делегирует эту

задачу классу Point, который уже знает, как задать точку. Таким образом, классу Creature не нужно беспокоиться о том, как будут реализованы такие вещи.

Подсказка

Хорошее практическое правило заключается в том, что каждый класс должен быть создан для выполнения одной задачи. Эта задача должна заключаться либо в хранении и обработке каких-либо данных (например, Point2D, std::string), либо в координации своих элементов (например, Creature). В идеале не в том и другом.

В нашем примере имеет смысл, что Creature не должно беспокоиться о том, как реализованы Points или как хранится имя. Задача Creature не в том, чтобы знать эти интимные детали. Задача Creature — беспокоиться о том, как координировать поток данных и следить за тем, чтобы каждый из участников класса знал, что он должен делать. Отдельные классы должны беспокоиться о том, как они будут это делать.

Агрегация

В предыдущем разделе — Композиция мы отметили, что композиция объектов — это процесс создания сложных объектов из более простых. Мы также говорили об одном из видов композиции объектов, который называется композицией. В композиционных отношениях весь объект отвечает за существование части.

В этом разделе мы рассмотрим другой тип композиции объектов, называемый агрегацией.

Чтобы считаться **совокупностью (агрегацией)**, целый объект и его части должны иметь следующие взаимосвязи:

- Часть (элемент) является частью объекта (класса)
- Часть (элемент) может (при желании) одновременно принадлежать более чем одному объекту (классу)
- Часть (элемент) не управляется объектом (классом)
- Часть (элемент) не знает о существовании объекта (класса)

Как и композиция, агрегация — это отношение «часть-целое», при котором части содержатся внутри целого, и это однонаправленное отношение. Однако, в отличие от композиции, части могут принадлежать более чем одному объекту одновременно, и целый объект не отвечает за существование и срок жизни частей. При создании агрегации она не отвечает за создание частей. При уничтожении агрегации она не отвечает за уничтожение частей.

Например, рассмотрим отношения между человеком и его домашним адресом. В этом примере для простоты мы будем считать, что у каждого человека есть адрес. Однако этот адрес может принадлежать более чем одному человеку одновременно: например, вам и вашему соседу по комнате или близкому человеку. Однако этот адрес не управляется человеком — он, вероятно, существовал до того, как человек туда переехал, и будет существовать после того, как человек уедет. Кроме того, человек знает, по какому адресу он живёт, но адреса не знают, какие люди там живут. Следовательно, это совокупное отношение.

В качестве альтернативы рассмотрим автомобиль и двигатель. Двигатель автомобиля является его частью. И хотя двигатель принадлежит автомобилю, он может принадлежать и другим вещам, например, человеку, которому принадлежит автомобиль. Автомобиль не отвечает за создание или разрушение двигателя. И хотя автомобиль знает, что у него есть двигатель (он должен знать, чтобы куда-то ехать), двигатель не знает, что он является частью автомобиля.

Когда речь идёт о моделировании физических объектов, использование термина «разрушен» может быть немного рискованным. Кто-то может возразить: «Если метеорит упал с неба и разбил машину, разве все детали машины тоже не будут разрушены?» Да, конечно. Но в этом виноват метеорит. Важно то, что машина не несёт ответственности за разрушение своих деталей (но это может быть вина внешней силы).

Можно сказать, что агрегатные модели отражают отношения «есть-у» (в отделе есть учителя, у автомобиля есть двигатель).

Как и в случае с составом, части совокупности могут быть единичными или мультипликативными.

Реализация агрегированных данных

Поскольку агрегации похожи на композиции в том, что они представляют собой отношения «часть-целое», они реализуются почти одинаково, и разница между ними в основном семантическая. В композиции мы обычно добавляем части в композицию с помощью обычных переменных-элементов (или указателей, где процесс выделения и освобождения памяти обрабатывается классом композиции).

В агрегации мы также добавляем части в качестве переменных-элементов. Однако эти переменные-элементы обычно являются либо ссылками, либо указателями, которые используются для указания на объекты, созданные за пределами области действия класса. Следовательно, агрегация обычно либо принимает объекты, на которые она будет указывать, в качестве параметров конструктора, либо начинается с пустого состояния, а подобъекты добавляются позже с помощью функций доступа или операторов.

Поскольку эти части существуют вне области видимости класса, при уничтожении класса переменная-элемент указателя или ссылки будет уничтожена (но не удалена). Следовательно, сами части будут существовать.

Давайте подробнее рассмотрим пример с учителем и кафедрой. В этом примере мы сделаем несколько упрощений: во-первых, на кафедре будет работать только один учитель. Во-вторых, учитель не будет знать, на какой кафедре он работает.

```
#include <iostream>
#include <string>
#include <string_view>

class Teacher
{
private:
    std::string m_name{};

public:
    Teacher(std::string_view name)
        : m_name{ name }
    {
    }

    const std::string& getName() const { return m_name; }
};

class Department
{
private:
    const Teacher& m_teacher; // This dept holds only one teacher for simplicity, but it
    could hold many teachers

public:
    Department(const Teacher& teacher)
        : m_teacher{ teacher }
    {
    }
};

int main()
{
    // Create a teacher outside the scope of the Department
    Teacher bob{ "Bob" }; // create a teacher

    {
        // Create a department and use the constructor parameter to pass
        // the teacher to it.
        Department department{ bob };

    } // department goes out of scope here and is destroyed

    // bob still exists here, but the department doesn't

    std::cout << bob.getName() << " still exists!\n";

    return 0;
}
```

В этом случае bob создаётся независимо от department и затем передаётся в конструктор department's. Когда department уничтожается, ссылка m_teacher уничтожается, но сам учитель не уничтожается, поэтому он продолжает существовать до тех пор, пока не будет уничтожен независимо от main() позже.

Выберите правильные отношения для того, что вы моделируете

Хотя в приведённом выше примере может показаться немного глупым, что учителя не знают, в каком отделе они работают, в контексте конкретной программы это может быть вполне нормально. Когда вы определяете, какие отношения реализовать, реализуйте самые простые отношения, которые соответствуют вашим потребностям, а не те, которые, как вам кажется, лучше всего подошли бы в реальном мире.

Например, если вы пишете симулятор автомастерской, вы можете захотеть реализовать автомобиль и двигатель как **совокупность (агрегацию)**, чтобы двигатель можно было снять и положить куда-нибудь на полку до следующего раза. Однако если вы пишете симулятор гонок, вы можете захотеть реализовать автомобиль и двигатель как композицию, поскольку в этом контексте двигатель никогда не будет существовать отдельно от автомобиля.

Наилучшая практика

Реализуйте самый простой тип отношений, который соответствует потребностям вашей программы, а не тот, который кажется правильным в реальной жизни.

Обобщающий состав и агрегирование

Композиции:

- Обычно используются обычные переменные-элементы
- Может использовать элементы-указатели, если класс сам обрабатывает выделение / освобождение объекта
- Ответственный за создание / разрушение деталей

Агрегации:

- Обычно используются указатели или ссылочные элементы, которые указывают на объекты или ссылаются на них, находящиеся за пределами области видимости класса-агрегата
- Не несет ответственности за создание / уничтожение деталей

Стоит отметить, что понятия композиции и агрегации можно свободно смешивать в рамках одного класса. Вполне возможно написать класс, который отвечает за создание/уничтожение одних частей, но не других. Например, у нашего класса «Кафедра» может быть название и преподаватель. Название, вероятно, будет добавляться в «Кафедру» путём композиции и создаваться и уничтожаться вместе с «Кафедрой». С другой стороны, преподаватель будет добавляться в «Кафедру» путём агрегации и создаваться/уничтожаться независимо.

Хотя агрегации могут быть чрезвычайно полезными, они также потенциально более опасны, поскольку агрегации не освобождают свои части. Освобождение частей возлагается на внешнюю сторону. Если у внешней стороны больше нет указателя или ссылки на освобождённые части или если она просто забудет выполнить очистку (при условии, что класс справится с этим), то произойдёт утечка памяти.

По этой причине предпочтение следует отдавать композициям, а не агрегациям.

Несколько предупреждений / ошибок

По ряду исторических и контекстуальных причин, в отличие от композиции, определение агрегации не является точным, поэтому в других справочных материалах вы можете встретить его определение, отличное от нашего. Это нормально, просто имейте это в виду.

И последнее замечание: на лекции — Введение в структуры, элементы и выбор элементов мы определили агрегированные типы данных (такие как структуры и классы) как типы данных, объединяющие несколько переменных. Вы также можете столкнуться с термином совокупный класс в вашем опыте работы с C++ это определяется как структура или класс, у которых нет встроенных конструкторов, деструкторов или перегруженных операторов присваивания, есть все открытые элементы и которые не используют наследование — по сути, это обычная структура данных. Несмотря на схожесть названий, агрегаты и агрегация — это разные понятия, и их не следует путать.

std::reference_wrapper

В приведённом выше примере Department/Teacher мы использовали ссылку в Department для хранения Teacher. Это работает нормально, если есть только один Teacher, но что, если в отделе несколько преподавателей? Мы хотели бы хранить этих преподавателей в каком-нибудь списке (например, в std::vector), но фиксированные массивы и различные списки стандартной библиотеки не могут содержать ссылки (потому что элементы списка должны быть присваиваемыми, а ссылки нельзя переназначать).

```
std::vector<const Teacher&> m_teachers{}; // Illegal
```

Вместо ссылок мы могли бы использовать указатели, но это открыло бы возможность хранить или передавать нулевые указатели. В примере Department/Teacher мы не хотим допускать нулевые указатели. Чтобы решить эту проблему, есть std::reference_wrapper.

По сути, std::reference_wrapper — это класс, который работает как ссылка, но также позволяет присваивать значения и копировать, поэтому он совместим со списками, такими как std::vector.

Хорошая новость в том, что вам не обязательно понимать, как это работает, чтобы использовать это. Вам нужно знать только три вещи:

1. std::reference_wrapper живет в заголовке <functional>.
2. Когда вы создаёте объект в std::reference_wrapper-оболочке, он не может быть анонимным (поскольку анонимные объекты имеют область видимости выражения, и это приведёт к потере ссылки).
3. Когда вы хотите вернуть свой объект обратно из std::reference_wrapper, вы используете get() функцию-элемент.

Вот пример использования std::reference_wrapper в std::vector:

```
#include <functional> // std::reference_wrapper
#include <iostream>
#include <vector>
#include <string>

int main()
{
    std::string tom{ "Tom" };
    std::string berta{ "Berta" };

    std::vector<std::reference_wrapper<std::string>> names{ tom, berta }; // these
strings are stored by reference, not value

    std::string jim{ "Jim" };

    names.emplace_back(jim);

    for (auto name : names)
    {
        // Use the get() member function to get the referenced string.
        name.get() += " Beam";
    }

    std::cout << jim << '\n'; // prints Jim Beam

    return 0;
}
```

В результате работы программы будет выведено:

```
Jim Beam
```

Чтобы создать вектор константных ссылок, нам нужно добавить const перед std::string вот так

```
// Vector of const references to std::string
std::vector<std::reference_wrapper<const std::string>> names{ tom, berta };
```

Ассоциация

В предыдущих двух разделах мы рассмотрели два типа композиции объектов: композицию и агрегацию. Композиция объектов используется для моделирования отношений, при которых сложный объект состоит из одного или нескольких более простых объектов (частей).

В этом уроке мы рассмотрим более слабый тип связи между двумя не связанными между собой объектами, который называется ассоциацией. В отличие от отношений, связанных с композицией объектов, в ассоциации нет подразумеваемых отношений «целое/часть».

Чтобы считаться ассоциацией, объект и другой объект должны быть связаны следующим образом:

- Связанный объект (элемент) в остальном не связан с объектом (классом).
- Связанный объект (элемент) может одновременно принадлежать более чем одному объекту (классу)
- Связанный объект (элемент) не управляется объектом (классом)
- Связанный объект (элемент) может знать или не знать о существовании объекта (класса)

В отличие от композиции или агрегации, где часть является частью целого объекта, в ассоциации связанный объект не имеет отношения к объекту. Как и в случае с агрегацией, связанный объект может одновременно принадлежать нескольким объектам и не управляется этими объектами. Однако, в отличие от агрегации, где связь всегда односторонняя, в ассоциации связь может быть односторонней или двусторонней (когда два объекта знают друг о друге).

Отношения между врачами и пациентами — отличный пример ассоциации. У врача явно есть отношения с пациентами, но концептуально это не отношения части и целого (объектной композиции). Врач может принимать много пациентов в день, а пациент может посещать многих врачей (возможно, он хочет получить второе мнение или ходит к врачам разных специальностей). Продолжительность жизни ни одного из объектов не связана с продолжительностью жизни другого.

Можно сказать, что модели ассоциации — это отношения «использует-а». Врач «использует» пациента (для получения дохода). Пациент использует врача (для любых необходимых ему медицинских целей).

Ассоциации - исполнители

Поскольку ассоциации — это широкий тип отношений, они могут быть реализованы множеством различных способов. Однако чаще всего ассоциации реализуются с помощью указателей, когда объект указывает на связанный с ним объект.

В этом примере мы реализуем двусторонние отношения между врачом и пациентом, поскольку врачам имеет смысл знать, кто их пациенты, и наоборот.

```
#include <functional> // reference_wrapper
#include <iostream>
#include <string>
#include <string_view>
#include <vector>

// Since Doctor and Patient have a circular dependency, we're going to forward declare Patient
class Patient;

class Doctor
{
private:
    std::string m_name{};
    std::vector<std::reference_wrapper<const Patient>> m_patient{};

public:
    Doctor(std::string_view name) :
        m_name{ name }
```

```

    {
    }

    void addPatient(Patient& patient);

    // We'll implement this function below Patient since we need Patient to be
    defined at that point
    friend std::ostream& operator<<(std::ostream& out, const Doctor& doctor);

    const std::string& getName() const { return m_name; }
};

class Patient
{
private:
    std::string m_name{};
    std::vector<std::reference_wrapper<const Doctor>> m_doctor{}; // so that we can
    use it here

    // We're going to make addDoctor private because we don't want the public to use
    it.
    // They should use Doctor::addPatient() instead, which is publicly exposed
    void addDoctor(const Doctor& doctor)
    {
        m_doctor.push_back(doctor);
    }

public:
    Patient(std::string_view name)
        : m_name{ name }
    {
    }

    // We'll implement this function below to parallel operator<<(std::ostream&,
    const Doctor&)
    friend std::ostream& operator<<(std::ostream& out, const Patient& patient);

    const std::string& getName() const { return m_name; }

    // We'll friend Doctor::addPatient() so it can access the private function
    Patient::addDoctor()
    friend void Doctor::addPatient(Patient& patient);
};

void Doctor::addPatient(Patient& patient)
{
    // Our doctor will add this patient
    m_patient.push_back(patient);

    // and the patient will also add this doctor
    patient.addDoctor(*this);
}

std::ostream& operator<<(std::ostream& out, const Doctor& doctor)
{
    if (doctor.m_patient.empty())
    {
        out << doctor.m_name << " has no patients right now";
        return out;
    }

    out << doctor.m_name << " is seeing patients: ";
    for (const auto& patient : doctor.m_patient)

```

```

        out << patient.get().getName() << ' ';

    return out;
}

std::ostream& operator<<(std::ostream& out, const Patient& patient)
{
    if (patient.m_doctor.empty())
    {
        out << patient.getName() << " has no doctors right now";
        return out;
    }

    out << patient.m_name << " is seeing doctors: ";
    for (const auto& doctor : patient.m_doctor)
        out << doctor.get().getName() << ' ';

    return out;
}

int main()
{
    // Create a Patient outside the scope of the Doctor
    Patient dave{ "Dave" };
    Patient frank{ "Frank" };
    Patient betsy{ "Betsy" };

    Doctor james{ "James" };
    Doctor scott{ "Scott" };

    james.addPatient(dave);

    scott.addPatient(dave);
    scott.addPatient(betsy);

    std::cout << james << '\n';
    std::cout << scott << '\n';
    std::cout << dave << '\n';
    std::cout << frank << '\n';
    std::cout << betsy << '\n';

    return 0;
}

```

Это печатает:

```

James is seeing patients: Dave
Scott is seeing patients: Dave Betsy
Dave is seeing doctors: James Scott
Frank has no doctors right now
Betsy is seeing doctors: Scott

```

В целом, вам следует избегать двунаправленных связей, если подойдёт однонаправленная, так как они усложняют работу и их сложнее писать без ошибок.

Рефлексивная ассоциация

Иногда объекты могут быть связаны с другими объектами того же типа. Это называется рефлексивной связью. Хорошим примером рефлексивной связи является связь между университетским курсом и его предпосылками (которые также являются университетскими курсами).

Рассмотрим упрощённый случай, когда у курса может быть только одно предварительное условие. Мы можем сделать что-то вроде этого:

```

#include <string>
#include <string_view>

class Course
{
private:
    std::string m_name{};
    const Course* m_prerequisite{};

public:
    Course(std::string_view name, const Course* prerequisite = nullptr):
        m_name{ name }, m_prerequisite{ prerequisite }
    {
    }
};

```

Это может привести к цепочке ассоциаций (у курса есть предварительное условие, у которого есть предварительное условие, и так далее...)

Ассоциации могут быть косвенными

Во всех предыдущих случаях мы использовали указатели или ссылки для непосредственного связывания объектов. Однако в случае ассоциации это не является обязательным. Достаточно любых данных, которые позволяют связать два объекта. В следующем примере мы покажем, как класс Driver может иметь однонаправленную ассоциацию с классом Car без использования указателя или ссылки на Car:

```

#include <iostream>
#include <string>
#include <string_view>

class Car
{
private:
    std::string m_name{};
    int m_id{};

public:
    Car(std::string_view name, int id)
        : m_name{ name }, m_id{ id }
    {
    }

    const std::string& getName() const { return m_name; }
    int getId() const { return m_id; }
};

// Our CarLot is essentially just a static array of Cars and a lookup function to
// retrieve them.
// Because it's static, we don't need to allocate an object of type CarLot to use it
namespace CarLot
{
    Car carLot[4] { { "Prius", 4 }, { "Corolla", 17 }, { "Accord", 84 }, { "Matrix", 62 } };

    Car* getCar(int id)
    {
        for (auto& car : carLot)
        {
            if (car.getId() == id)
            {
                return &car;
            }
        }
    }
}

```

```

        }
    }

    return nullptr;
}

};

class Driver
{
private:
    std::string m_name{};
    int m_carId{}; // we're associated with the Car by ID rather than pointer

public:
    Driver(std::string_view name, int carId)
        : m_name{ name }, m_carId{ carId }
    {
    }

    const std::string& getName() const { return m_name; }
    int getCarId() const { return m_carId; }
};

int main()
{
    Driver d{ "Franz", 17 }; // Franz is driving the car with ID 17

    Car* car{ CarLot::getCar(d.getCarId()) }; // Get that car from the car lot

    if (car)
        std::cout << d.getName() << " is driving a " << car->getName() << '\n';
    else
        std::cout << d.getName() << " couldn't find his car\n";

    return 0;
}

```

Данная программа выводит следующее:

```
Franz is driving a Corolla
```

В приведённом выше примере у нас есть CarLot, в котором хранятся наши автомобили. У водителя, которому нужен автомобиль, нет ссылки на свой автомобиль — вместо этого у него есть идентификатор автомобиля, который мы можем использовать, чтобы получить автомобиль из CarLot, когда он нам понадобится.

В этом конкретном примере делать так — немного глупо, поскольку для получения автомобиля из CarLot требуется неэффективный поиск (указатель, соединяющий их, работает намного быстрее). Однако есть преимущества в использовании уникального идентификатора вместо указателя. Например, вы можете ссылаться на объекты, которые в данный момент не находятся в памяти (возможно, они находятся в файле или в базе данных и могут быть загружены по запросу). Кроме того, указатели могут занимать 4 или 8 байт. Если пространство ограничено, а количество уникальных объектов невелико, то ссылка на них с помощью 8-битного или 16-битного целого числа может сэкономить много памяти.

Сводка по композиции, агрегации и ассоциации

Вот сводная таблица, которая поможет вам запомнить разницу между композицией, агрегацией и ассоциацией:

	Композиция	Агрегация	Ассоциация
Тип отношения	Целое/часть	Целое/часть	В остальном не связанные
Элементы могут принадлежать к нескольким классам	НЕТ	ДА	ДА
Существование участников, управляемое классом	ДА	НЕТ	НЕТ
Направленность	Однонаправленный	Однонаправленный	Однонаправленный или двунаправленный
Глагол отношения	Часть-из	Состоит	Использует

Зависимости

На данный момент мы рассмотрели 3 типа связей: составные, агрегатные и ассоциативные. Самый простой тип мы оставили напоследок: зависимости.

В повседневной речи мы используем термин «зависимость», чтобы указать на то, что один объект зависит от другого в выполнении определённой задачи. Например, если вы сломаете ногу, то будете зависеть от костылей, чтобы передвигаться (но не наоборот). Цветы зависят от пчёл, которые опыляют их, чтобы они плодоносили или размножались (но не наоборот).

Зависимость возникает, когда один объект вызывает функцию другого объекта для выполнения какой-либо конкретной задачи. Это более слабая связь, чем ассоциация, но всё же любое изменение в объекте, от которого зависит другой объект, может нарушить функциональность (зависимого) вызывающего объекта. Зависимость — это всегда односторонняя связь.

Хорошим примером зависимости, которую вы уже много раз видели, является `std::ostream`. Наши классы, использующие `std::ostream`, используют его для выполнения задачи по выводу чего-либо на консоль, но не более того.

Например:

```
#include <iostream>

class Point
{
private:
    double m_x{};
    double m_y{};
    double m_z{};

public:
    Point(double x=0.0, double y=0.0, double z=0.0): m_x{x}, m_y{y}, m_z{z}
    {
    }

    friend std::ostream& operator<< (std::ostream& out, const Point& point); // Point
has a dependency on std::ostream here
};

std::ostream& operator<< (std::ostream& out, const Point& point)
{
    // Since operator<< is a friend of the Point class, we can access Point's members
    directly.
    out << "Point(" << point.m_x << ", " << point.m_y << ", " << point.m_z << ')';

    return out;
}

int main()
{
    Point point1 { 2.0, 3.0, 4.0 };

    std::cout << point1; // the program has a dependency on std::cout here

    return 0;
}
```

В приведенном выше коде `Point` не имеет прямого отношения к `std::ostream`, но он зависит от `std::ostream`, поскольку `operator<<` использует `std::ostream` для вывода `Point` на консоль.

Зависимости против ассоциации в C++

Обычно возникает некоторая путаница в том, что отличает зависимость от ассоциации.

В C++ ассоциации — это отношения, при которых один класс содержит прямую или косвенную «ссылку» на связанный с ним класс в качестве элемента. Например, класс «Доктор» содержит массив указателей на своих пациентов в качестве элемента. Вы всегда можете спросить у Доктора, кто его пациенты. Класс «Водитель» содержит идентификатор автомобиля, которым владеет объект «Водитель», в качестве целочисленного элемента. Водитель всегда знает, какой автомобиль с ним связан.

Зависимости обычно не являются элементами. Скорее, объект, от которого зависит, обычно создается по мере необходимости (например, при открытии файла для записи данных) или передается в функцию в качестве параметра (например, `std::ostream` в перегруженном операторе `<<` выше).